Repeater Experiment

1. Overview

This experiment implements a **full-duplex I2S audio recorder/player based on ESP32-S3**. It drives both the ES8311 ADC and DAC via a full-duplex I2S bus, supporting dual-mode operation: long-press to record and short-press to play. Recording data is stored in SPIFFS in WAV format, with a three-stage pop-noise suppression mechanism to ensure playback quality.

```
[I2S RX] ← [ES8311 ADC] ← [MEMS MIC]
[I2S TX] → [ES8311 DAC] → [NS4150 PA] → [SPK]
                            ↕
                     [SPIFFS /record.wav]
```

Function	Description
Recording	ES8311 ADC, 16kHz / 16-bit / mono, max 20 seconds, 640KB PSRAM buffer
Playback	ES8311 DAC, 16kHz / 16-bit / stereo (mono duplicated), SPIFFS storage
Button	BOOT button (GPIO0): long-press+hold=record, release=stop; short-press=play/pause/resume
LED	WS2812 (RMT driven): recording=green, done=red, playing=rainbow animation
Pop Suppression	Three-stage: mute+zero+flush+PA on → restore volume+unmute → flush+mute

---

2. Core Concepts

2.1 Full-Duplex I2S

Full-duplex means I2S opens both TX and RX channels simultaneously, sharing MCLK/BCLK/WS clock signals. In full-duplex mode, TX and RX sampling parameters must match (same sample rate, same bit width) because they share the WS clock. In Arduino's `ESP_I2S` library, `begin()` creates the TX channel, and `configureRX()` appends the RX channel and enables hardware STEREO→MONO mixing—the data read from RX is already mono, so no software extraction of the left channel is needed.

```
          MCLK (GPIO15)          DOUT (GPIO8)
ESP32 ──────────────────→ ES8311 ──────────────────→ ES8311 SDIN (DAC)
          BCLK (GPIO16)          DIN  (GPIO17)
          WS   (GPIO18)  ESP32 ←────────────────── ES8311 SDOUT (ADC)
                         I2C  (GPIO41/42)
                    ESP32 ←──────────────────→ ES8311 Control Interface
```

2.2 Pop-Noise Suppression Mechanism

Improper startup sequencing between the ES8311 DAC and NS4150 amplifier can produce a loud "pop" sound at power-on. This project uses a **three-stage strategy**:

```
Stage 1 (_popInit): Mute DAC → Gain to zero → 1 second zero-value I2S flush → PA on (speaker input is 0V)
Stage 2 (_popStart): Set PLAY_VOLUME → Unmute → 10ms delay → 512 bytes zero-value preamble
Stage 3 (_popStop): 512 bytes zero-value suffix → Re-mute DAC
```

Core idea: **At the moment the amplifier powers on, the speaker receives a silent signal**, so the pop is naturally cut off. When pausing, immediately mute the DAC; when resuming, unmute first then clear the flag.

2.3 SPIFFS File Storage

Recording data is stored in WAV format at `/record.wav`, mounted with a single line `SPIFFS.begin(true)`. `WavRecorder` writes a placeholder header then appends PCM, and on close it backfills fileSize and dataSize; `WavPlayer` parses the header then returns PCM in chunks. Recording strategy: first accumulate 640KB in PSRAM, then write to SPIFFS in one shot after stopping—avoids SPIFFS random write latency affecting real-time performance.

---

3. Hardware Features and Dependencies

3.1 ESP32-S3 Audio Hardware

Feature	Parameter
MCU	ESP32-S3R8 (8MB PSRAM built-in), 16MB Flash
I2S	2 channels, standard/left-aligned/right-aligned/DSP
I2C	2 channels, standard/fast mode
RMT	8 TX/RX channels, 80MHz resolution (WS2812 driver)
Arduino Core	v3.3.8 (based on IDF v5.x)

3.2 ES8311 Specifications

Feature	Parameter
I2C Address	0x18 (7-bit)
ADC/DAC SNR	93dBA / 95dBA
Sample Rate	8kHz ~ 96kHz
MIC PGA	0~42dB, 6dB/step
Working Mode	ADC / DAC / BOTH (this project uses BOTH)

ES8311 is in slave mode, BCLK/LRCLK are provided by ESP32, MCLK = 16000 × 256 = 4.096MHz. I2S Philips standard format, 16-bit, stereo slots.

3.3 Pin Assignment

Pin	Function	Connection Target
GPIO42/41	I2C SCL/SDA	ES8311
GPIO15	I2S MCLK	ES8311 MCLK
GPIO16	I2S BCLK	ES8311 BCLK

Pin	Function	Connection Target
GPIO18	I2S WS	ES8311 LRCK
GPIO8	I2S DOUT	ES8311 SDIN (DAC)
GPIO17	I2S DIN	ES8311 SDOUT (ADC)
GPIO7	PA_EN	NS4150 amplifier
GPIO48	WS2812 Data	RGB LED
GPIO0	BOOT button	Active low, internal pull-up

3.4 IDE Configuration and Library Dependencies

Configuration	Value
Board	ESP32S3 Dev Module
USB CDC On Boot	Enabled
Flash Size	16MB (128Mb)
Partition Scheme	Custom (`partitions.csv`)
PSRAM	OPI PSRAM

Library	Purpose	Reference Tutorial
ESP32 (by Espressif)	Arduino core (includes ESP_I2S, SPIFFS, Wire, FreeRTOS)	Arduino → 1. Before Development → 1. Before Development → Environment Setup (Arduino-ESP32 Environment Setup)

ES8311 driver is fully hand-written, WS2812 uses ESP-IDF native RMT API, no additional LED library needed. Recording file is approximately 640KB, custom partition table required (4MB app + 11.6MB SPIFFS).

---

4. Program Flow

```
setup():
  1. Wire.begin(41,42) → codec.begin(16000,true,true) → setMicGain(40) → setMute(true)
  2. audio.begin():
     ① PA off → ② RMT+LED → ③ I2S full-duplex+MCLK → ④ Pop suppression (mute+flush+PA on)
     ⑤ SPIFFS mount → ⑥ Alloc 640KB recording buffer → ⑦ Delete old recording → ⑧ Button
task (50Hz)

loop(): Print status every 10s

taskButton(): Debounce 30ms → Timer (50ms granularity)
  ├─ Long press (≥800ms) → Record: xTaskCreate(taskRecord) → Wait for release →
_recStop=true
```

```
  └─ Short press (<800ms) → Play/Pause/Resume

taskRecord(): LED green → i2s.readBytes→PSRAM → Buffer full/stop → WavRecorder→SPIFFS → LED
red
taskPlay():   WavPlayer→_popStart()→f.read→mono→stereo→i2s.write→rainbow→_popStop()
```

Recording data flow: ADC → I2S DIN(GPIO17) → I2S RX (hardware STEREO→MONO) → PSRAM (640KB) → WavRecorder → SPIFFS

Playback data flow: SPIFFS → WavPlayer → mono→stereo copy → I2S DOUT(GPIO8) → DAC → PA → Speaker

File structure:

```
Arduino_Recorder_Player/
├── Arduino_Recorder_Player.ino  ← Main entry
├── ES8311Driver.h / .cpp        ← ES8311 register-level driver
├── AudioSystem.h / .cpp         ← I2S full-duplex + button + LED + pop suppression
├── WavHandler.h / .cpp          ← WAV read/write (WavRecorder+WavPlayer)
└── partitions.csv               ← Partition table
```

---

5. Source Code Details

Full source code: `5. Source code → Arduino → 4.Audio → Arduino_Recorder_Player → Arduino_Recorder_Player.ino`

5.1 Main Entry (.ino)

`setup()` chains I2C→Codec→AudioSystem initialization, `loop()` only does periodic status printing. All core logic runs in AudioSystem's FreeRTOS tasks.

```
#include <Wire.h>
#include "ES8311Driver.h"
#include "AudioSystem.h"

ES8311Driver  codec;
AudioSystem   audio(codec);        // AudioSystem references codec, doesn't own it

void setup() {
    Serial.begin(115200); delay(800);
    Serial.println("\n===== ES8311 Audio Recorder & Player v2 =====");

    Wire.begin(41, 42);            // Initialize I2C bus (SDA=41, SCL=42)
    Wire.setClock(100000);         // 100kHz standard mode

    // I2C device scan
    Serial.print("[SYS] I2C scan:");
    for (uint8_t a = 1; a < 127; a++) {
        Wire.beginTransmission(a);
        if (Wire.endTransmission() == 0) Serial.printf("  0x%02X", a);
    }
```

```
    if (!codec.begin(16000, true, true)) { // ADC+DAC both enabled
        Serial.println("[SYS] FATAL: ES8311 init failed!");
        while (1) delay(1000);
    }
    codec.setMicGain(40);                  // Microphone gain 40dB
    codec.setMute(true);                   // Initialize muted
    audio.begin();                         // Full audio system init

    Serial.println("\n===== System Ready =====");
}

void loop() {
    static unsigned long last = 0;
    if (millis() - last > 10000) {         // Print status every 10s
        last = millis();
        Serial.printf("[SYS] %s\n",
            audio.isRecording() ? "Recording..." :
            audio.isPlaying()   ? (audio.isPaused() ? "Paused" : "Playing...") : "Idle");
    }
    delay(1000);
}
```

5.2 ES8311Driver —— Register-Level Driver

Since there is no equivalent to `esp_codec_dev` library in the Arduino ecosystem, this project hand-wrote a complete ES8311 register-level driver.

Key Registers

Register	Address	Description
RST	0x00	Reset control + clock gating
CLK1~CLK8	0x01~0x08	Clock management (MCLK divider/OSR/BCLK/LRCK)
SDPIN/SDPOUT	0x09/0x0A	DAC/ADC I2S format
ADCG	0x16	ADC PGA gain (0~7 steps)
DACPWR	0x31	DAC mute bit[6:5]: 11=mute, 00=normal
DACV	0x32	DAC digital volume (0x00~0xFF)
DACS2	0x37	DAC ramp soft-start (pop prevention)

Class Interface

`ES8311Driver` encapsulates I2C register read/write (*w/r*), clock tree, power-up sequence, and gain/volume control. `setGainReg()` is a special "backdoor" for pop suppression—it bypasses the 75% safety limit and zero-value protection, directly writing to the `DACV` register.

```
class ES8311Driver {
    static const uint8_t ADDR = 0x18;   // I2C 7-bit address
```

```
    bool _ok;          // Init success
    bool _adc;         // ADC path enabled
    bool _dac;         // DAC path enabled
    uint32_t _rate;    // Current sample rate

    void _w(uint8_t r, uint8_t v);           // I2C write register
    uint8_t _r(uint8_t r);                   // I2C read register
    void _open();                            // Clock tree + ALC + I/O init
    void _setSampleRate(uint32_t rate);      // Sample rate dividers
    void _setI2SFormat();                    // I2S standard 16-bit format
    void _powerUp();                         // Power-up sequence
    void _powerDown();                       // Power-down sequence

public:
    ES8311Driver();
    bool begin(uint32_t rate=16000, bool adc=true, bool dac=true); // Init
    void setMicGain(uint8_t db);             // MIC gain 0~42dB, 6dB/step
    void setVolume(uint8_t pct);             // Volume 0~75% (safety limit)
    void setMute(bool m);                    // DAC mute toggle
    void setGainReg(uint8_t val);            // Direct write DACV (pop suppression)
    bool isOK() const;                       // Init success check
};
```

begin() Flow

`begin()` executes in strict order: I2C probe → clock tree → sample rate divider → I2S format → power-up sequence.

```
bool ES8311Driver::begin(uint32_t sampleRate, bool adcEnable, bool dacEnable) {
    _adc = adcEnable; _dac = dacEnable; _rate = sampleRate;
    // I2C probe: send address, check ACK
    Wire.beginTransmission(ADDR);
    if (Wire.endTransmission() != 0) {
        Serial.printf("[ES8311] ERROR: device not found at 0x%02X!\n", ADDR);
        return false;
    }
    _open();                                 // Clock tree init
    _setSampleRate(_rate);                   // Sample rate divider
    _setI2SFormat();                         // I2S standard 16-bit format
    _powerUp();                              // Power-up sequence
    _ok = true;
    _dumpRegs();                             // Debug output
    return true;
}
```

Clock Tree (*open) and Power-Up Sequence (*powerUp)

`_open()` configures MCLK, OSR, and sample rate (SR=31 → Fs=16kHz), GPIO double-write for I2C noise immunity. `_powerUp()` executes power-up sequence: release clock gate → analog wake-up → VMID soft-start → DAC ramp soft-start.

```
void ES8311Driver::_open() {
    _w(CLK1,0x30);  // MCLK div=1, BCLK/ADCDAC=slave in
    _w(CLK2,0x00);  // pre_div=1, multi=x1
    _w(CLK3,0x10);  // fs_mode=0, ADC_OSR=16
    _w(CLK4,0x10);  // DAC_OSR=16
    _w(CLK5,0x00);  // ADC_div=1, DAC_div=1
    _w(ADCG,0x24);  // ADC PGA gain 24dB
    _w(PGA ,0x00);  // PGA gain 0dB, differential input
    _w(TIMER,0x00); // Disable internal timer
    _w(SR,  0x1F);  // Sample rate 16kHz (SR=31→Fs=MCLK/256)
    _w(INT1,0x7F);  // Clear all interrupt flags
    _w(RST ,0x80);  // bit7=1 release register reset
    _w(CLK1,0x3F);  // bit5=1 enable MCLK input
    _w(GPIO,0x08);  // GPIO1=internal ADC reference
    _w(GPIO,0x08);  // Double-write for I2C noise immunity
    _w(FILT,0x10);  // ADC anti-alias filter
    _w(ALC2,0x0A);  // ALC noise gate -26.0dB, no compression
    _w(ALC3,0x6A);  // ALC limiter 95.05%, recovery time 0.67s
}

void ES8311Driver::_powerUp() {
    _w(RST, 0x80);   // Release register reset
    _w(CLK1, 0x3F);  // MCLK buffer on, div=1, BCLK=slave
    _w(SDPIN,  _r(SDPIN) &0xBF);
    _w(SDPOUT, _r(SDPOUT)&0xBF);
    if (_adc) _w(ADC1, 0xBF);       // ADC digital volume max
    _w(SWR,  0x02);                 // LDAC+LADC+LADCref analog wake-up
    _w(INT2, 0x00);                 // Clear interrupts
    _w(HPF, 0x1A);                  // HPF fc=0.13Hz, SEQ=1
    _w(PWR,  0x01);                 // VMID soft-start 13k, analog power on
    _w(ADCPWR,0x40);                // ADC analog power on
    _w(DACS2,0x08);                 // DAC ramp soft-start
    _w(GP45,0x00);                  // GPIO2=Hi-Z
}
```

Gain and Volume

`setMicGain()` uses nested conditional expressions for dB→step mapping, `setVolume()` uses Arduino `map()` for linear mapping, `setMute()` does read-modify-write on DACPWR preserving other bits.

```
void ES8311Driver::setMicGain(uint8_t db) {
    if (!_ok || !_adc) return;
    // Step mapping: 6dB per step
    uint8_t v = db<6?0: db<12?1: db<18?2: db<24?3: db<30?4: db<36?5: db<42?6: 7;
    _w(ADCG, v);
}

void ES8311Driver::setVolume(uint8_t pct) {
    if (!_ok || !_dac) return;
    if (pct > 75) pct = 75;
    uint8_t v = pct ? map(pct, 1, 100, 50, 255) : 0;
    _w(DACV, v);
```

```
}

void ES8311Driver::setMute(bool m) {
    if (!_ok || !_dac) return;
    uint8_t v = _r(DACPWR) & 0x9F;  // Keep bit[7,4:0]
    if (m) v |= 0x60;                // bit[6:5]=11 → mute
    _w(DACPWR, v);
}

void ES8311Driver::setGainReg(uint8_t val) {
    if (!_ok || !_dac) return;
    _w(DACV, val);
}
```

5.3 AudioSystem —— Audio System Core

Manages I2S full-duplex, three-stage pop suppression, WS2812 LED (direct RMT drive), button state machine, and record/playback tasks.

5.3.1 Header Global Definitions

Pin macros and audio parameters are defined centrally in `AudioSystem.h`. `REC_BUF_LEN = 16000 × 20 × 2 = 640000` —the exact buffer capacity for 20 seconds of 16kHz 16-bit mono recording.

```
// I2S Pins
#define PIN_MCLK    15       // Master clock output 4.096MHz
#define PIN_BCLK    16       // Bit clock 512kHz
#define PIN_WS      18       // Word select 16kHz
#define PIN_DOUT    8        // Data out (playback -> ES8311 DAC)
#define PIN_DIN     17       // Data in (ES8311 ADC -> ESP32)

// Control Pins
#define PIN_PA      7        // PA enable HIGH=on
#define PIN_BTN     0        // BOOT button, pull-up, active LOW
#define PIN_RGB     48       // WS2812 data line

// Audio Parameters
#define SAMPLE_RATE     16000           // Sample rate 16kHz
#define PLAY_VOLUME     75              // Playback volume 0~75%
#define REC_BUF_LEN     (16000*20*2)    // Recording buffer 640KB = 20s
#define FRAME_SIZE      1920            // I2S read frame size ~60ms
#define LONG_PRESS_MS   800             // Long press threshold 800ms
```

`AudioSystem` is the core module of the recorder/player. `volatile` state variables are shared across tasks (written by buttons, read by record/playback/main loop), and `rmt_channel_handle_t` uses ESP-IDF native API directly.

```
class AudioSystem {
    ES8311Driver     &_codec;      // Reference to codec driver
    I2SClass          _i2s;        // I2S full-duplex instance
```

```
    volatile bool    _playing   = false;  // Playback active
    volatile bool    _paused    = false;  // Paused
    volatile bool    _recording = false;  // Recording active
    volatile bool    _recStop   = false;  // Stop recording request
    volatile uint32_t _recBytes = 0;     // Recorded bytes count

    int16_t  *_recBuf  = nullptr;        // Recording PSRAM buffer 640KB
    uint16_t  _hue     = 0;              // Rainbow hue 0-65535

    rmt_channel_handle_t _rmtChan = nullptr;  // RMT TX channel
    rmt_encoder_handle_t _rmtEnc  = nullptr;  // RMT passthrough encoder

    void _pa(bool on);            void _ledInit();          // PA control + LED init
    void _ledSet(uint8_t r,g,b);  void _ledRainbowStep();   // Solid color + HSV rainbow
step
    void _i2sInit();              void _popInit/_popStart/_popStop(); // I2S + three-stage
pop
    static void _s_rec(void*p);   static void _s_play(void*p); static void _s_btn(void*p);
    void taskRecord();            void taskPlay();          void taskButton();
public:
    AudioSystem(ES8311Driver &c) : _codec(c) {}
    void begin();  // 8-step total init
    bool isRecording() const; bool isPlaying() const; bool isPaused() const;
};
```

5.3.2 I2S Full-Duplex Initialization

`begin()` starts TX+MCLK, `configureRX()` appends RX and enables hardware STEREO→MONO conversion—the data read is already mono, no software channel extraction needed.

```
void AudioSystem::_i2sInit() {
    _i2s.setPins(PIN_BCLK, PIN_WS, PIN_DOUT, PIN_DIN, PIN_MCLK);

    // MCLK must start before ES8311 init — ES8311 uses external MCLK as clock source
    _i2s.begin(I2S_MODE_STD, SAMPLE_RATE,
               I2S_DATA_BIT_WIDTH_16BIT,              // 16-bit sample depth
               I2S_SLOT_MODE_STEREO,                  // Stereo slot (2x16bit=32bit/frame)
               I2S_STD_SLOT_BOTH);                    // Both channels active

    _i2s.configureRX(SAMPLE_RATE, I2S_DATA_BIT_WIDTH_16BIT,
                     I2S_SLOT_MODE_STEREO,
                     I2S_RX_TRANSFORM_16_STEREO_TO_MONO); // HW mix, no SW merge
}
```

5.3.3 Three-Stage Pop Suppression

`_popInit` allocates 64KB zero-value buffer from PSRAM and flushes for 1 second—pipeline is all silence when PA turns on. `_popStart` **restores volume first, then unmutes**—reversing the order causes transition noise. `_popStop` **flushes first, then mutes**—ensures no residual audio in pipeline when muted.

```
void AudioSystem::_popInit() {
    _codec.setMute(true);                              // 1. Mute throughout
```

```
    _codec.setGainReg(0x00);                          // 2. DAC volume to 0

    // 3. Alloc zero buffer, flush I2S pipe
    int16_t *z = (int16_t *)heap_caps_calloc(1, SAMPLE_RATE * 4,
                        MALLOC_CAP_SPIRAM | MALLOC_CAP_8BIT);  // Prefer PSRAM
    if (!z)
        z = (int16_t *)heap_caps_calloc(1, SAMPLE_RATE * 4,
                        MALLOC_CAP_INTERNAL | MALLOC_CAP_8BIT); // Fallback
    if (z) {
        _i2s.write((uint8_t *)z, SAMPLE_RATE * 4);   // Write 1s zeros, clear pipe
        free(z);
    }
    _pa(true); delay(100);                            // 4. PA on, no pop
}

void AudioSystem::_popStart() {
    _codec.setVolume(PLAY_VOLUME);                    // Restore playback volume
    _codec.setMute(false); delay(10);                 // Unmute, wait DAC settle

    int16_t z[512];                                   // ~8ms @ 16kHz stereo
    memset(z, 0, sizeof(z));
    _i2s.write((uint8_t *)z, sizeof(z));              // Flush last silent frame
}

void AudioSystem::_popStop() {
    int16_t z[512];
    memset(z, 0, sizeof(z));
    _i2s.write((uint8_t *)z, sizeof(z));              // Flush pipe first
    _codec.setMute(true);                             // Then mute DAC
}
```

5.3.4 Recording Task

First accumulate 640KB in PSRAM (I2S real-time read + memcpy), then write to SPIFFS in one shot after stopping—avoids SPIFFS random write latency affecting real-time performance. Buffer is allocated on first use then reused, `delay(5)` yields CPU each round.

```
void AudioSystem::taskRecord() {
    unsigned long t0 = millis();

    // First alloc PSRAM buffer (640KB), reuse thereafter
    if (!_recBuf) {
        _recBuf = (int16_t *)heap_caps_malloc(REC_BUF_LEN, MALLOC_CAP_SPIRAM |
MALLOC_CAP_8BIT);
        if (!_recBuf)
            _recBuf = (int16_t *)heap_caps_malloc(REC_BUF_LEN, MALLOC_CAP_INTERNAL |
MALLOC_CAP_8BIT);
        if (!_recBuf) { /* FATAL */ _recording = false; vTaskDelete(NULL); return; }
    }
    _recBytes = 0;
    _ledSet(0, 255, 0);                               // Green = recording
```

```
    uint8_t *rx = (uint8_t *)malloc(FRAME_SIZE);     // I2S read frame buffer
    if (!rx) { _recording = false; vTaskDelete(NULL); return; }

    while (!_recStop) {
        uint32_t rem = REC_BUF_LEN - _recBytes;      // Remaining space
        if (rem < FRAME_SIZE) break;                  // Buffer full, auto-stop

        size_t rd = _i2s.readBytes((char *)rx, FRAME_SIZE);
        if (rd > 0) {
            memcpy((uint8_t *)_recBuf + _recBytes, rx, rd); // Append to PSRAM
            _recBytes += rd;
        }
        delay(5);                                     // Yield
    }
    _recStop = false; free(rx);

    if (_recBytes > 0) {
        WavRecorder wav;
        if (wav.open("/record.wav", SAMPLE_RATE, 16, 1)) {
            wav.write((uint8_t *)_recBuf, _recBytes);  // PSRAM → SPIFFS
            wav.close();                                // Patch WAV header
        }
    }
    _ledSet(0, 0, 0); delay(100);                    // Off transition
    _ledSet(255, 0, 0);                              // Red = done
    _recording = false;
    vTaskDelete(NULL);                               // Self-delete
}
```

5.3.5 Playback Task

`_popStart()` restores volume then unmutes first, then enters playback loop: read mono from SPIFFS → copy to stereo → `_i2s.write()`, `delay(3)` controls write rate close to 16kHz theoretical value, prevents reading at full speed.

```
void AudioSystem::taskPlay() {
    unsigned long t0 = millis();
    WavPlayer wav;
    if (!wav.open("/record.wav")) { _playing = false; vTaskDelete(NULL); return; }

    _popStart();                                     // Pop suppression → PA on

    uint8_t  mono[1024];                             // Mono read buffer
    int16_t  stereo[1024];                           // Stereo output buffer
    uint32_t remain = wav.dataSize(), sent = 0;     // Remaining/sent bytes

    while (remain && _playing) {
        while (_paused) delay(50);                   // Pause gate: busy-wait

        size_t rd = wav.read(mono, sizeof(mono));    // Read from SPIFFS
        if (!rd) break;
```

```
        // Mono → Stereo expand: duplicate each sample to both channels
        size_t n = rd / 2;                              // Sample count
        for (size_t i = 0; i < n; i++) {
            stereo[i * 2]     = ((int16_t *)mono)[i];  // Left
            stereo[i * 2 + 1] = ((int16_t *)mono)[i];  // Right
        }
        _i2s.write((uint8_t *)stereo, n * 4);       // n samples × 4 bytes/stereo_sample
        remain -= rd; sent += rd;
        _ledRainbowStep();                              // Rainbow step per frame
        delay(3);                                       // Throttle write rate
    }

    _popStop();                                         // Pop suppression → PA off
    wav.close();
    _playing = false; _paused = false;
    _ledSet(0, 0, 0);                                  // LED off
    vTaskDelete(NULL);
}
```

5.3.6 Button Task

Debounce 30ms → hold timer (50ms granularity) → ≥800ms long press to record / <800ms short press to play. When unpausing, **unmute first then clear the flag**—ensures DAC is ready before audio reaches it. `_s_rec` / `_s_play` / `_s_btn` are static trampolines—FreeRTOS requires `void task(void*)` signature, C++ member functions have implicit `this` incompatibility, forwarded via `void*`.

```
void AudioSystem::taskButton() {
    pinMode(PIN_BTN, INPUT_PULLUP);
    Serial.println("=== Ready ===");
    Serial.println("[BTN] Long press+hold=Record | Short press=Play/Pause/Resume");

    for (;;) {
        if (digitalRead(PIN_BTN) == LOW) {          // Detect falling edge
            delay(30);                              // Debounce
            if (digitalRead(PIN_BTN) != LOW) continue; // Bounce, ignore

            // Measure hold duration
            int hold = 0;
            while (digitalRead(PIN_BTN) == LOW) {
                delay(50); hold += 50;              // 50ms granularity
                if (hold >= LONG_PRESS_MS) break;   // Threshold reached
            }

            if (hold >= LONG_PRESS_MS) {            // Long press → Record
                if (_playing||_recording) { /* Reject */ }
                else {
                    _recording = true; _recStop = false;
                    xTaskCreate(_s_rec, "rec", 8192, this, 5, NULL);

                    while (digitalRead(PIN_BTN) == LOW) delay(50); // Wait release
                    _recStop = true;                // Signal stop
                    delay(300);                     // Wait cleanup
```

```
                    while (_recording) delay(10);    // Confirm exit
                }
            } else {                                  // Short → Play/Pause
                if (_playing) {
                    if (_paused) {
                        _codec.setMute(false);       // Unmute
                        _paused = false;             // Resume
                    } else {
                        _codec.setMute(true);        // Mute (keep stream)
                        _paused = true;              // Pause
                    }
                } else {
                    if (!SPIFFS.exists("/record.wav"))
                        Serial.println("[BTN] No recording file!");
                    else {
                        _playing = true; _paused = false;
                        xTaskCreate(_s_play, "play", 8192, this, 5, NULL);
                    }
                }
                while (digitalRead(PIN_BTN) == LOW) delay(10); // Wait release
            }
        }
        delay(20);                                   // 50Hz polling
    }
}
```

5.3.7 System Total Initialization (begin)

8 strict ordered steps: PA off must be before `_popInit` (which turns PA on internally), I2S+MCLK must be before `_popInit` (depends on I2S writing zeros).

```
void AudioSystem::begin() {
    _pa(false);                                      // 1. PA off
    _ledInit();                                      // 2. LED init
    _i2sInit();                                      // 3. I2S start + MCLK out
    _popInit();                                      // 4. Pop suppression init

    SPIFFS.begin(true);                              // 5. Mount SPIFFS (formatOnFail=true)

    // 6. Alloc record buffer (PSRAM first, fallback internal RAM)
    _recBuf = (int16_t *)heap_caps_malloc(REC_BUF_LEN, MALLOC_CAP_SPIRAM | MALLOC_CAP_8BIT);
    if (!_recBuf)
        _recBuf = (int16_t *)heap_caps_malloc(REC_BUF_LEN, MALLOC_CAP_INTERNAL |
MALLOC_CAP_8BIT);
    if (!_recBuf)
        Serial.printf("[SYS] WARN: Recording buffer alloc failed (%dKB)\n", REC_BUF_LEN /
1024);
    else
        Serial.printf("[SYS] Recording buffer: %dKB (%ds max)\n",
                      REC_BUF_LEN / 1024, REC_BUF_LEN / (SAMPLE_RATE * 2));

    // 7. Remove leftover recording
```

```
    if (SPIFFS.exists("/record.wav")) {
        SPIFFS.remove("/record.wav");
        Serial.println("[SYS] Deleted old recording");
    }

    xTaskCreate(_s_btn, "btn", 4096, this, 5, NULL);
}
```

---

6. Operation Flow

6.1 IDE Configuration and Compilation

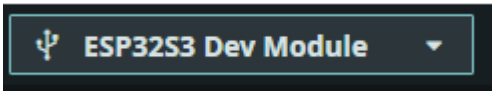
Step 1: Install Dependencies

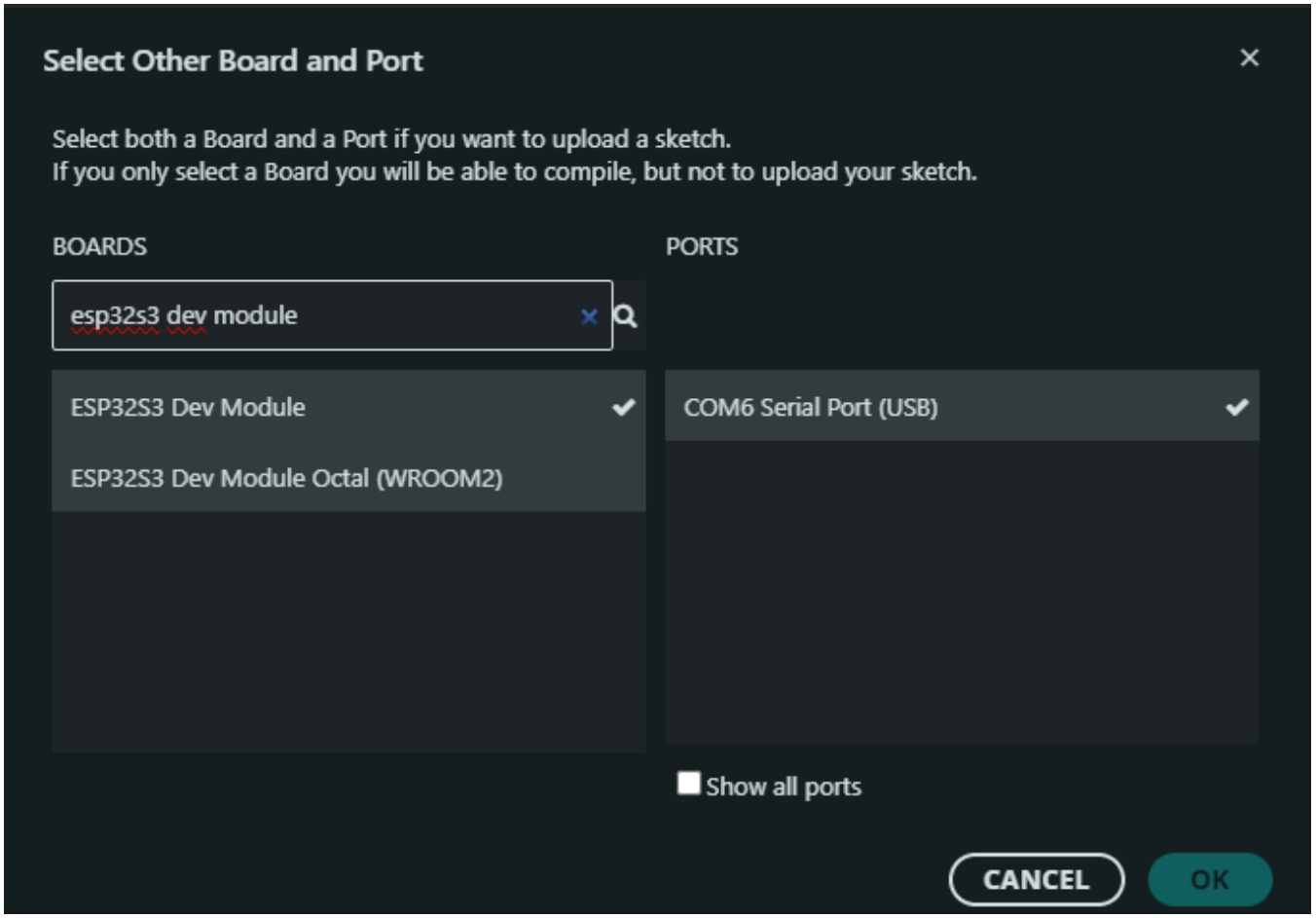
- Install "esp32 by Espressif Systems" (v3.3.8+) from board manager. ESP_I2S, SPIFFS, Wire, etc. are all built-in core libraries, no additional installation needed.

Step 2: Select Configuration

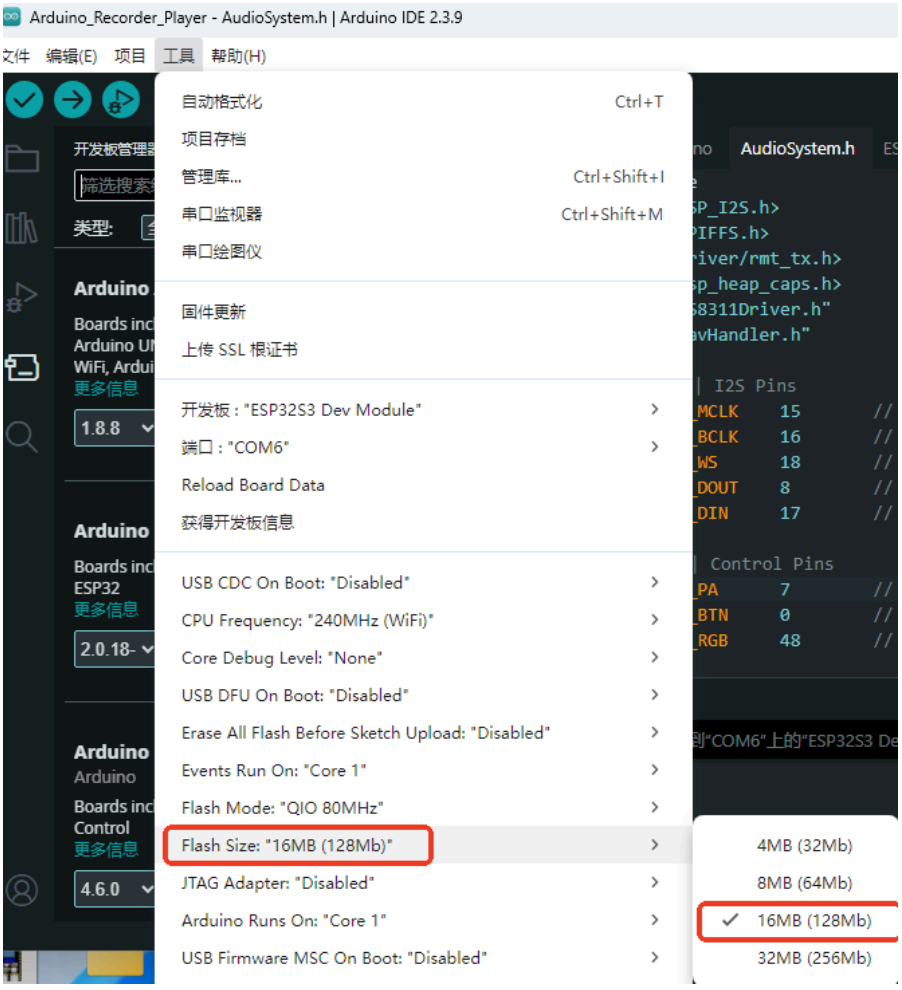
Configuration	Value
Board	ESP32S3 Dev Module
Flash Size	16MB (128Mb)
Partition Scheme	Custom (`partitions.csv`)
PSRAM	OPI PSRAM

Board Selection

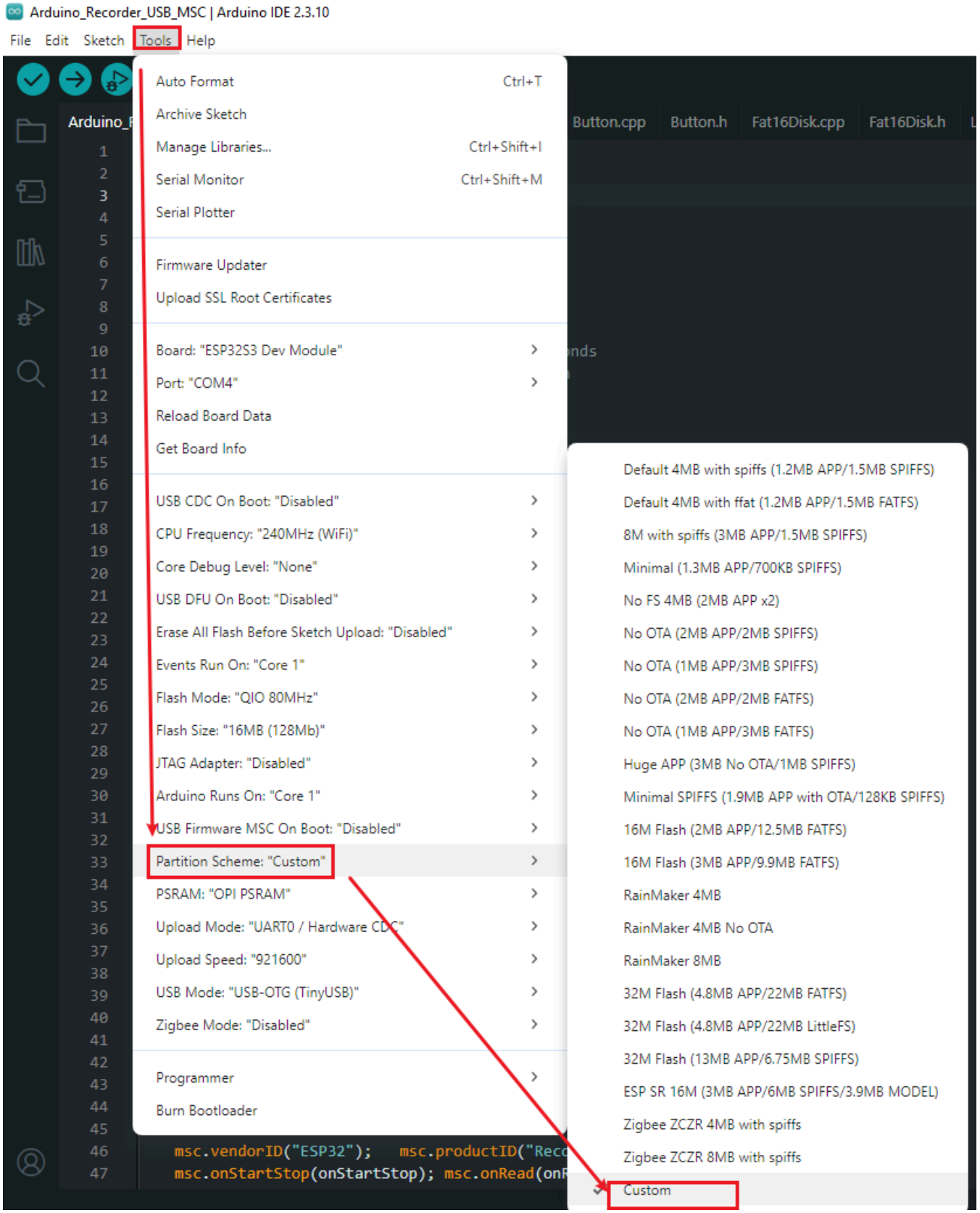




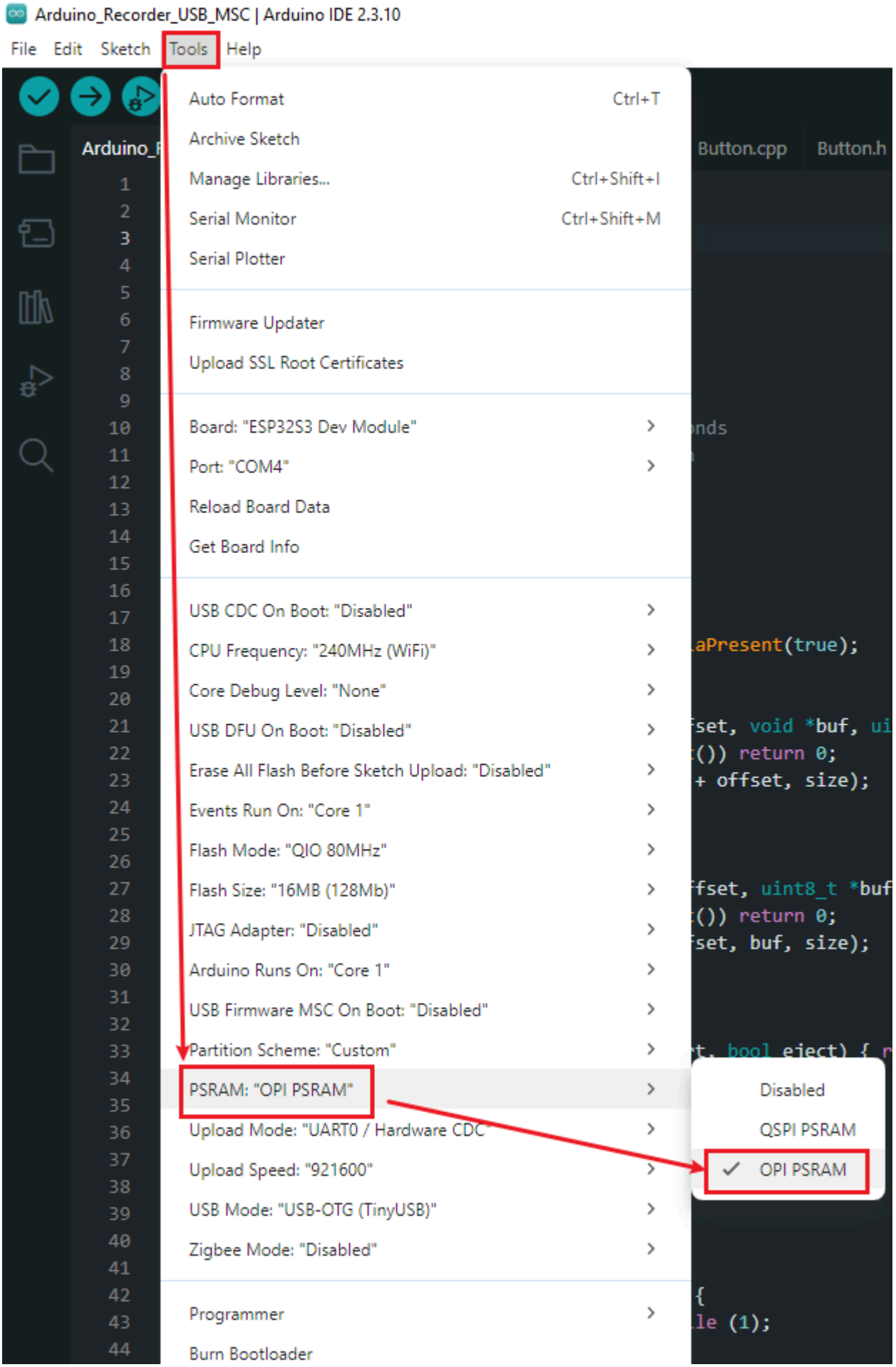
Enable Flash Size: 16MB (128Mb)



Custom Partition:



Enable PSRAM:



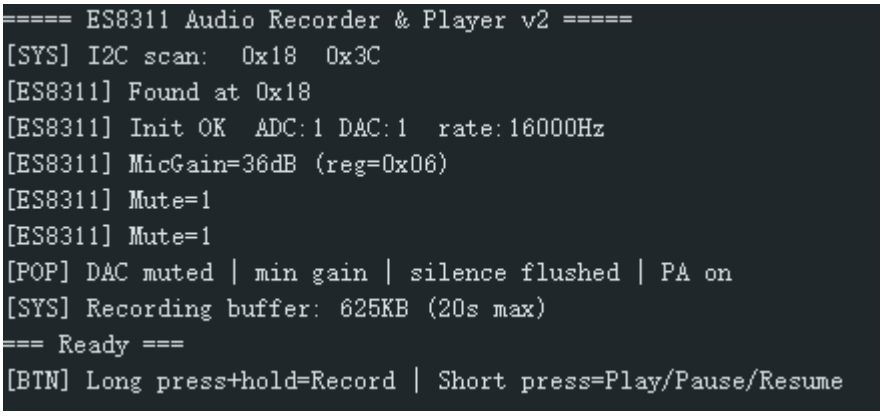
Step 3: Build and Flash

Arduino flashing flow: See `Arduino → 1. Before Development → 2. Build and Flash`.

> Recording file is approximately 640KB, **custom partition table required** (`partitions.csv`, 4MB app + 11.6MB SPIFFS). Built-in partition schemes (e.g., Minimal SPIFFS only 190KB) have insufficient capacity.

6.2 Expected Operation

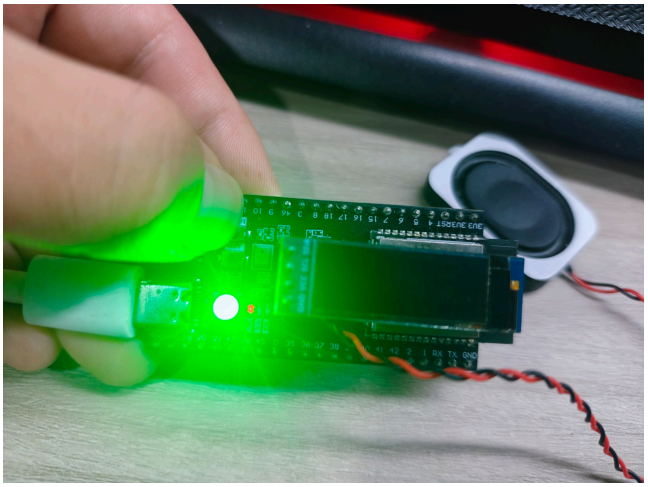
Power-on serial output:



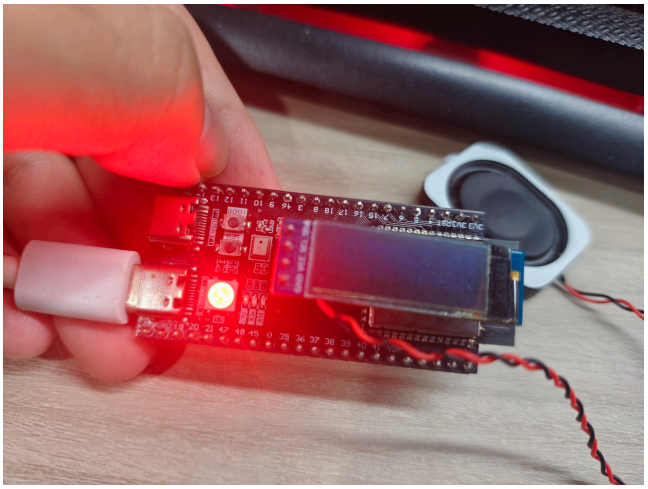

```
===== ES8311 Audio Recorder & Player v2 =====
[SYS] I2C scan:  0x18  0x3C
[ES8311] Found at 0x18
[ES8311] Init OK  ADC:1 DAC:1  rate:16000Hz
[ES8311] MicGain=36dB (reg=0x06)
[ES8311] Mute=1
[POP] DAC muted | min gain | silence flushed | PA on
[SYS] Recording buffer: 625KB (20s max)
=== Ready ===
[BTN] Long press+hold=Record | Short press=Play/Pause/Resume
```

Operation	Expected	LED
Long press+hold (≥0.8s)	Start recording	Green
Release	Stop and save	Red→off
Short press (<0.8s)	Play	Rainbow
Short press during playback	Pause	Off
Short press when paused	Resume	Rainbow
End of playback	Auto stop	Off
Short press with no file	Print "No recording file!"	—

Recording:



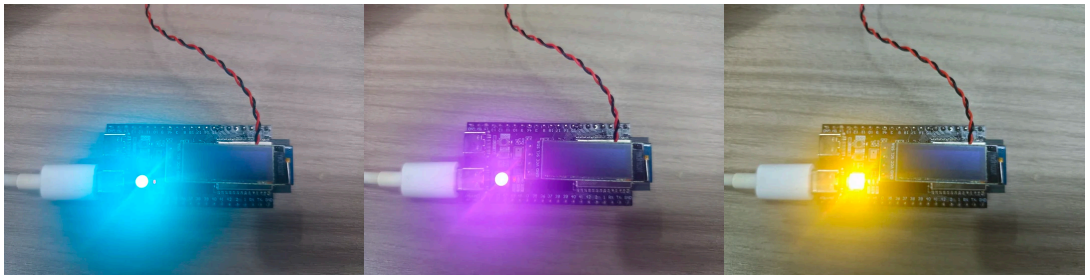
Save:



Serial Print:

```
===== System Ready =====
[BTN] Long press -> Start recording
[REC] START
[REC] Max 20s  Buf:625KB
[BTN] Release -> Stop recording
[WAV] Recording started: 16000Hz 16bit 1ch -> /record.wav
[WAV] Saved: 159360 bytes PCM data
[REC] STOP 159360B 4896ms -> 32549 B/s (expect 32000)
```

Playback:



Playback Print:

```
[WAV] File: 16000Hz 1ch 16bit  PCM=159360 bytes
[ES8311] Volume=70% (reg=0xC0)
[ES8311] Mute=0
```

Pause:

```
[BTN] Short press -> Pause
[ES8311] Mute=1
```

Resume:

```
[BTN] Short press -> Resume
[ES8311] Mute=0
```

Playback End:

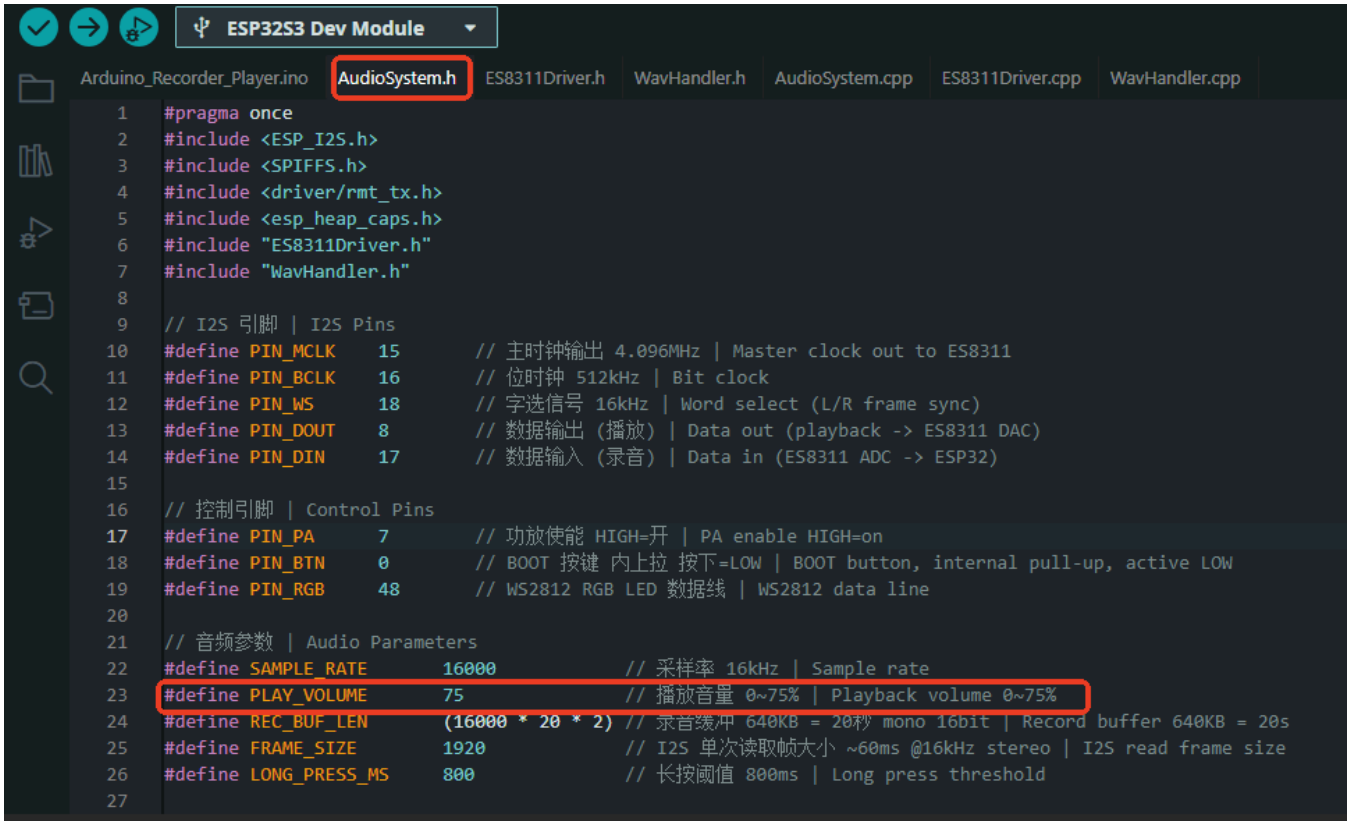
```
[ES8311] Mute=1
[PLAY] DONE 301440B 28945ms -> 10414 B/s (expect 32000)
```

6.3 Edge Cases

Scenario	Expected Behavior
Short press with no recording file	Print "No recording file!"
Long press during playback	Print "Playing, cannot record!", reject
Recording buffer full (20s)	Auto stop
No operation at power-on	Idle wait, PA on but DAC muted
PSRAM allocation failed	Fallback to internal RAM; full failure prints WARN, recording unavailable but no crash
ES8311 not found	Print FATAL then infinite loop

6.4 Volume Adjustment

Playback volume is determined by the `PLAY_VOLUME` macro in `AudioSystem.h`, default value 75. In simple terms, the larger this number, the louder the speaker; 0 is mute—recompile and flash to hear the difference.



Why is the limit 75, not 100? The ES8311 chip has an internal "digital volume knob" called the `DACV` register, and turning it all the way up is 255. The `setVolume()` function converts your percentage to a 0~255 range (e.g., 75% maps to approximately 191). But this chip has a flaw—turning it past 75% causes sound to "break" and become harsh and distorted. So the code simply cuts it off: no matter what value you pass, anything over 75 is treated as 75. This is not a bug—it's intentional audio protection.

If you really feel the volume is not loud enough, don't modify the 75 limit—that's the chip's ceiling, and pushing past it only makes the sound worse. The correct approach is to adjust the hardware: the NS4150 amplifier chip has a gain resistor—replacing it can increase the overall volume by a level.

Where to change it? Open `AudioSystem.h`, around line 23:

```
#define PLAY_VOLUME     75              // Playback volume 0~75%
```

Change 75 to your desired value (any integer between 0 and 75), save, recompile and upload.

---

7. Common Issues

Symptom	Cause	Solution
No sound on playback/recording	MCLK not output / I2S clock error	Check GPIO15 for 4.096MHz
Pop sound on playback	Pop sequence not executed correctly	Confirm *popInit*→popStart→_popStop order
Recording all zeros	I2S DIN(GPIO17) not connected / ADC not enabled	Check codec.begin() second parameter

Symptom	Cause	Solution
Recording volume low	MIC gain insufficient	Increase setMicGain() (max 42dB)
Abnormal playback speed	WAV sample rate mismatch	Ensure 16kHz
SPIFFS space insufficient	Partition too small	Use custom partition CSV (11.6MB)
Button unresponsive	Debounce/polling parameters unsuitable	Increase delay(30) or decrease delay(20)
I2C scan no 0x18	ES8311 hardware issue	Check power and GPIO41/42
PSRAM allocation failed	PSRAM not enabled	Confirm Tools→PSRAM→OPI PSRAM
ESP_I2S.h not found	ESP32 Arduino core version too low	Update esp32 core ≥ v3.0.0

Appendix A: ES8311 Register Quick Reference (BOTH Mode)

Register	Address	Write Value	Function
RST	0x00	0x80	Release reset
CLK_MGR1	0x01	0x30→0x3F	Enable MCLK input
CLK_MGR2	0x02	0x00	pre_div=1, multi=x1
CLK_MGR3	0x03	0x10	fs_mode=0, ADC_OSR=16
CLK_MGR4	0x04	0x10→0x20	DAC_OSR=16→32
CLK_MGR6	0x06	bclk_div=4	BCLK = MCLK/4 = 1.024MHz
SDP_IN/OUT	0x09/0A	0x0C	I2S standard mode, 16bit
SYS_SR	0x10	0x1F	Fs=MCLK/256=16kHz
ADC_GAIN	0x16	0x06 (36dB)	PGA gain 0~7
SYS_SWR	0x0E	0x02	Analog wake-up
ADC_PWR	0x15	0x40	ADC analog power on
DAC_PWR	0x31	bit[6:5]	Mute (11=mute, 00=normal)
DAC_VOL	0x32	0xBF (75%)	Digital volume
DAC_S2	0x37	0x08	DAC ramp soft-start
GPIO_SEL	0x44	0x08→0x58	Enable internal reference

Appendix B: Code Index

File	Lines	Function
`Arduino_Recorder_Player.ino`	47	Main entry: setup()/loop()
`ES8311Driver.h`	70	Register enum + class declaration
`ES8311Driver.cpp`	156	I2C read/write + clock tree + power + gain/volume
`AudioSystem.h`	72	I2S full-duplex + LED + button + pop API
`AudioSystem.cpp`	333	Record/play/button tasks + pop suppression + LED
`WavHandler.h`	51	WAV header + WavRecorder/WavPlayer declaration
`WavHandler.cpp`	64	WAV create/patch + open/read
`partitions.csv`	5	Partition table (4MB app + 11.6MB SPIFFS)